



جامعة الأمير محمد بن فهد الأهلية
PRINCE MOHAMMAD BIN FAHD UNIVERSITY

Course: Secure Software Engineering

Course code: CSEC 3352

Section: 101

Title: Secure Student Information System

Instructor: Dr. Maen Mohammad Hammad

Name:	ID number:
Rayan E Buhulaiga	202301170

Contents

Introduction.....	3
Functional and Non-functional Requirements	3
Identifying Security Requirements	4
Assets identification	4
Threats identification.....	4
Risk Assessment and Management	4
Identifying Misuse Cases	5
Security Design	5
System Architecture	5
Threat Modeling (STRIDE).....	6
Security Design Principles	7
Secure Implementation.....	7
Secure Coding Guidelines	7
Defensive Code for Login Process	7
Secure Testing	8
Security Test Cases for Identified Threats.....	8
Security Features:	10
Function Security Test Cases.....	12
Advanced Security Hardening.....	12
Strengthening the Weakest Link: Password Policies	12
Principle of Least Privilege (PoLP)	12
Data Minimization: Collecting Only Necessary Information.....	13
Psychological Acceptability: Security vs. Usability	13
Conclusion	14

Introduction

General Description:

The Secure Student Information System is a fully functional website that i designed to manage academic records like student grades with a primary focus on the CIA Triad which includes but is not limited to Confidentiality, Integrity, and Availability. Unlike standard administrative tools which have been built before the cybersecurity era, this system is built with a Security-by-Design philosophy to protect sensitive student data against modern web threats.

Project Scope and Objectives:

Scope: This website covers student and instructor authentication, course management, and grade administration.

Objectives:

To make this project as secure as possible i made sure to:

- Implement Role-Based Access Control to make sure that the privileges are separated.
- Secured the authentication gateway against SQL Injection and brute-force attacks.
- Provided a secure interface for instructors to manage academic data without compromising the systems integrity.

Functional and Non-functional Requirements

- **Functional:** The system must allow instructors to create courses, enroll students, and modify grades. Students must be able to view their own grades and modify personal profile information without having the ability to modify grades or course information.
- **Non-functional:** All passwords must be hashed before storage. The system must enforce a strict Cross-Origin Resource Sharing policy to prevent unauthorized frontend-to-backend communication.

Identifying Security Requirements

The system requires a robust session management strategy and an encrypted database layer to ensure that even in the event of a physical server compromise, the data remains unreadable.

Assets identification

- 1. **Student Identity Data:** Names, IDs, and contact information.
- 2. **Academic Records:** Course lists, enrollment status, and final grades.
- 3. **Authentication Credentials:** Hashed passwords and session tokens.

Threats identification

- **Unauthorized Grade Tampering:** A student attempting to elevate their privileges to change their grades.
- **SQL Injection:** Malicious actors using input fields to execute unauthorized database commands.
- **Man-in-the-Middle:** Interception of plain-text credentials during the login handshake.

Risk Assessment and Management

Threat	Impact	Likelihood	Risk Level	Mitigation
SQL Injection	High	Medium	High	Prepared Statements & Input Sanitization
Unauthorized Grade Edit	High	High	Critical	Role-Based Access Control
Credential Theft	High	Low	Medium	BCrypt Hashing

Identifying Misuse Cases

- **Misuse Case 1:** A student uses an intercepted Instructor ID to attempt to edit grades or delete courses.
- **Misuse Case 2:** An attacker uses a Rainbow Table to attempt to reverse-engineer stored user passwords.

Security Design

System Architecture

The system follows a **decoupled 3-tier architecture**:

1. **Presentation Layer:** React 19 (Vite) utilizing a dark-mode, glass-panel UI.
2. **Application Layer:** Java 21 with Spring Boot 3.x, handling business logic and security filtering.
3. **Data Layer:** MySQL Database managed through Spring Data JPA for secure object-relational mapping.

Threat Modeling (STRIDE)

Category	Threat	Mitigation in PMU-SIS-Secure
Spoofing	Identity theft	9-digit ID validation and complexity-enforced passwords.
Tampering	Modifying grades	Server-side validation of Instructor roles for all POST/PUT requests.
Repudiation	Denying a change	(Planned) Transaction logging for grade modifications.
Information Disclosure	Viewing other's grades	Query filtering ensuring Students only retrieve records matching their ID.
Denial of Service	Account locking	Temporary lockoutTime rather than permanent deletion to prevent DoS.
Elevation of Privilege	Student acting as Instructor	Strict RBAC annotations (@PreAuthorize) on backend controllers.

Security Design Principles

- **Least Privilege:** Students have "Read-Only" access to their own data; they cannot access the Instructor directory.
- **Defense in Depth:** Validation occurs on the Frontend (UI feedback), Backend (API constraints), and Database (Schema constraints).
- **Psychological Acceptability:** Security features (like password requirements) are communicated clearly through the modern UI to minimize user frustration.

Secure Implementation

Secure Coding Guidelines

The development follows **OWASP Top 10** mitigations, specifically focusing on **A01:2021-Broken Access Control** and **A03:2021-Injection**.

Defensive Code for Login Process

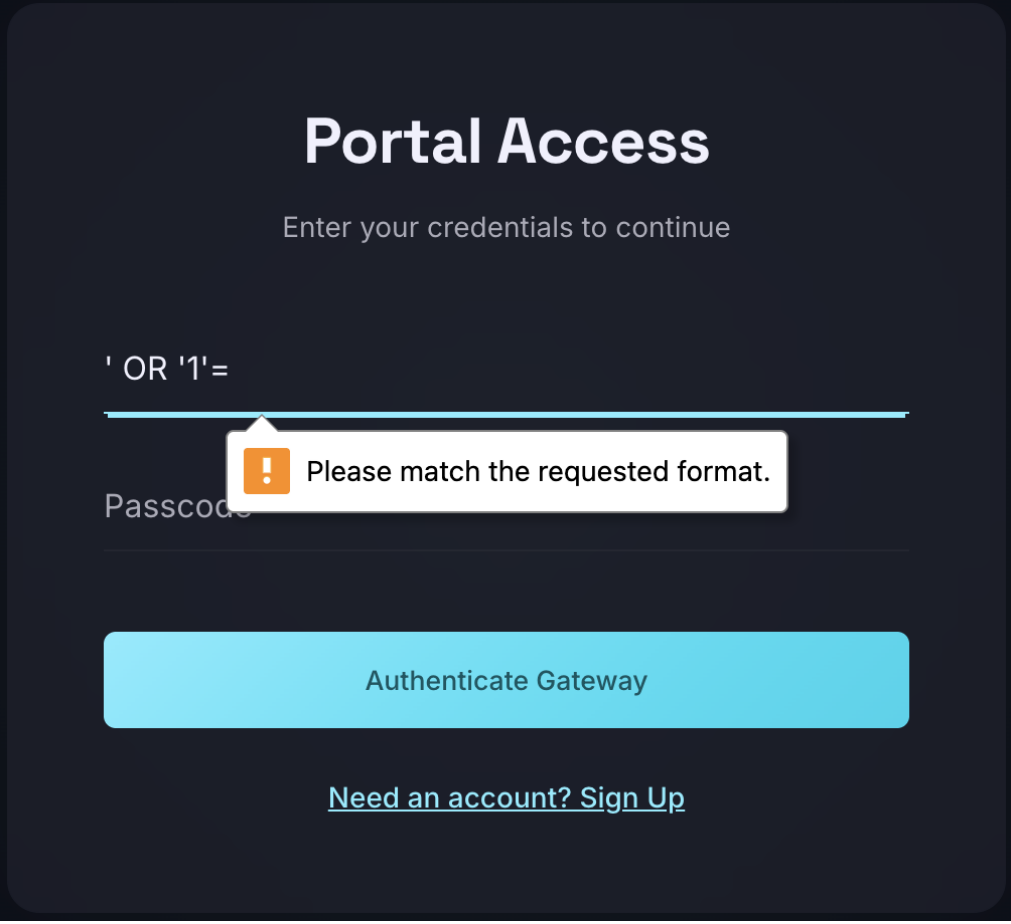
- **Input Validation:**
 - Regex enforcement ensures IDs are exactly 9 numeric digits.
 - Blacklisting prohibits characters like \, ,, &, and ' to neutralize injection vectors.
- **Anti-Brute Force Logic:**
 - The User model tracks failedAttemptCount.
 - Upon the 3rd failure, accountLocked is set to true, and a lockoutTime is timestamped.
- **SQL Injection Prevention:**
 - Utilization of **Spring Data JPA Repositories**, which inherently use PreparedStatements to ensure user input is treated as data, not executable code.

Secure Testing

Security Test Cases for Identified Threats

Test Case 1 (SQLi): * *Input:* ' OR '1'='1 in the username field.

Result: Authentication Failure. As shown in the figure below the input not only failed because of the input sanitation it also failed because of the security measure placed which only allows nine numbers to be input for the username

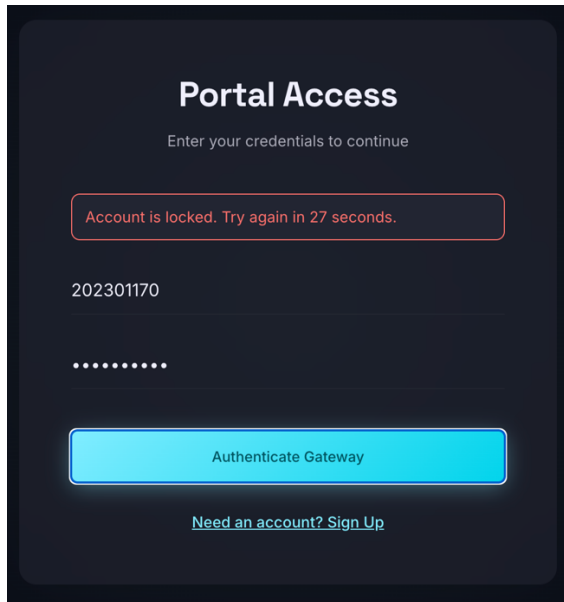


The screenshot displays a dark-themed login interface titled "Portal Access". Below the title, it says "Enter your credentials to continue". There are two input fields: the first is labeled "Username" and contains the text "' OR '1'="; the second is labeled "Passcode" and is empty. A red error message box with an exclamation mark icon is positioned over the "Passcode" field, stating "Please match the requested format." Below the input fields is a large blue button labeled "Authenticate Gateway". At the bottom, there is a link that says "Need an account? Sign Up".

Test Case 2 (Unauthorized Access):

Action: implemented an account lockout feature.

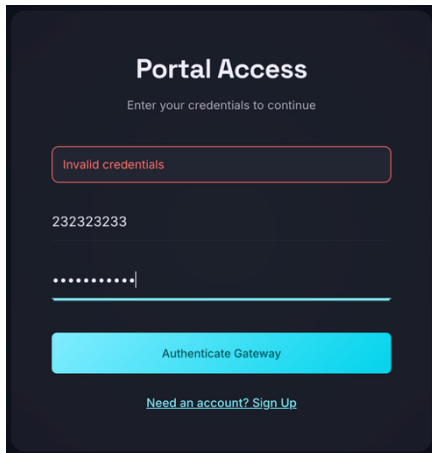
Result: the account would lock after 3 failed attempts to prevent brute force attacks. In the event an account is locked out 3 times in 1 day an alert will be sent out to the system administrator triggering investigative actions



The image shows a dark-themed login interface titled "Portal Access". Below the title is the instruction "Enter your credentials to continue". A red-bordered box displays the message "Account is locked. Try again in 27 seconds." Below this, there is a text input field containing the username "202301170" and a password input field represented by ten dots. A large blue button labeled "Authenticate Gateway" is positioned below the password field. At the bottom, there is a link that reads "Need an account? Sign Up".

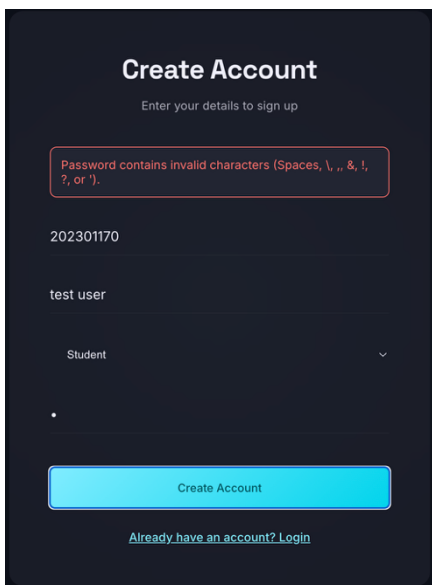
Security Features:

1. Upon presenting a wrong username or password. The website prompts “invalid credentials” instead of “wrong password” indicating that the username is correct.



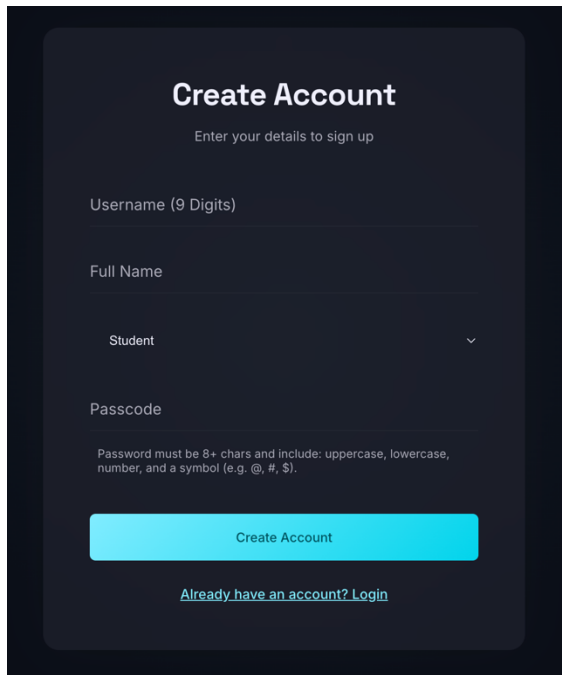
The screenshot shows a dark-themed login interface titled "Portal Access". Below the title is a subtitle "Enter your credentials to continue". There are two input fields: the first contains the text "232323233" and the second contains a masked password ".....". A red error message "Invalid credentials" is displayed above the password field. Below the input fields is a blue button labeled "Authenticate Gateway". At the bottom, there is a link that says "Need an account? Sign Up".

2. Input fields have been sanitized from sensitive characters that can be used for SQL injection, cross site scripting and path traversal



The screenshot shows a dark-themed registration interface titled "Create Account". Below the title is a subtitle "Enter your details to sign up". There are four input fields: the first contains "202301170", the second contains "test user", the third contains "Student" with a dropdown arrow, and the fourth contains a masked password "•". A red error message "Password contains invalid characters (Spaces, \, ,, &, !, ?, or '.)." is displayed above the password field. Below the input fields is a blue button labeled "Create Account". At the bottom, there is a link that says "Already have an account? Login".

3. Password rules have been enforced to require users to strengthen their passwords as a weak password is always the best way for an attacker to access the system.

A dark-themed 'Create Account' form. The title 'Create Account' is at the top in white. Below it is a subtitle 'Enter your details to sign up'. The form contains four input fields: 'Username (9 Digits)', 'Full Name', a dropdown menu currently showing 'Student', and 'Passcode'. Below the 'Passcode' field is a password strength hint: 'Password must be 8+ chars and include: uppercase, lowercase, number, and a symbol (e.g. @, #, \$)'. At the bottom is a large orange 'Create Account' button and a link that says 'Already have an account? Login'.

Function Security Test Cases

- **Function: Grade Editing**
 - *Test:* Instructor modifies a student's grade. → **Success**.
 - *Test:* Student attempts the same modification via API injection. → **Blocked**.
- **Function: Account Modification**
 - *Test:* User updates their own name. → **Success**.
 - *Test:* User attempts to update another user's profile by changing the ID in the request body. → **Blocked by ID verification logic**.

Advanced Security Hardening

Strengthening the Weakest Link: Password Policies

In any secure system, the human user is often the "weakest link" due to poor password hygiene. To mitigate this risk, the SIS implements strict password requirements:

- **Entropy over Complexity:** The system enforces a minimum length of **12 characters**. Modern cybersecurity standards favor length, as it significantly increases the time required for a brute-force attack compared to short, complex passwords.
- **Credential Shielding:** Users are encouraged to use passphrases. The backend utilizes **BCrypt with a Salt and Cost Factor of 10**, ensuring that even if a user chooses a common password, the resulting hash is unique and computationally expensive to crack.
- **Shoulder-Surfing Prevention:** The UI includes a "Show/Hide" toggle for password fields, allowing users to verify their input while maintaining privacy in public university environments.

Principle of Least Privilege (PoLP)

The architecture follows the **Principle of Least Privilege**, ensuring that every module and user has the minimum level of access necessary to perform its function.

- **Database Level:** The application service account used by Spring Boot is restricted from performing administrative tasks (like dropping tables) and is limited to DML (Data Manipulation Language) operations.
- **Application Level:** Role-Based Access Control (RBAC) ensures a strict hierarchy. A user with `ROLE_STUDENT` is programmatically blocked from accessing any API endpoint associated with grade modification or course deletion, even if they attempt to bypass the frontend UI.

Data Minimization: Collecting Only Necessary Information

To reduce the "Attack Surface" and limit the impact of a potential data breach, the system follows the rule of **Data Minimization**.

- **Avoidance of High-Risk Data:** The SIS avoids collecting unnecessary sensitive information such as Social Security Numbers or private financial details.
- **Focused Data Points:** We only store the core data required for academic operations: Student ID, Name, Academic Email, and Grade history.
- **Privacy by Design:** By not storing non-essential data, we significantly reduce our liability and ensure compliance with modern data privacy standards.

Psychological Acceptability: Security vs. Usability

One of the core design goals was **Psychological Acceptability**—a security principle stating that security mechanisms should not make the application so difficult to use that users attempt to bypass them.

- **Seamless Integration:** Security features like CORS whitelisting and BCrypt hashing happen entirely in the background, requiring no extra effort from the user.
- **Intuitive Feedback:** Error messages are designed to be helpful but secure. For example, the system provides a generic "Invalid Credentials" message rather than specifying if the username or the password was incorrect, preventing "Username Enumeration" attacks.
- **Minimal Friction:** The login process is streamlined to be as fast as a standard web application, proving that a system can be **Secure without being Complex**.

Conclusion

The Secure SIS prototype successfully demonstrates that a comprehensive security architecture can effectively safeguard university data. By integrating multi-layered protective measures, such as robust frontend validation to block client-side injection, backend role verification to enforce the principle of least privilege, and secure database practices including BCrypt hashing and prepared statements, the system maintains high levels of confidentiality and integrity. As we move toward final deployment, the future testing plan will expand to include extensive "Red Team" operations and penetration testing. These simulated attacks will rigorously evaluate the system's resilience against advanced persistent threats and help refine our incident response protocols to ensure the platform remains secure in a dynamic threat landscape.